ASEN 5264 Decision Making under Uncertainty
Homework 6: POMDPs

April 20, 2026

1 Exercises

Question 1. (20 pts)
Write the following three elements of a POMDP solver that uses the QMDP approximation:

1. A belief `Updater` and the associated `update` function to calculate the Bayesian belief update[1].
2. A `Policy struct` and associated `action` function that chooses an action based on a list of alpha vectors.
3. A function that calculates the QMDP alpha vectors for a `POMDP` and returns them as an object of the `Policy` type described above.

The starter code contains the skeleton code for these three items. Use your QMDP code and the SARSOP.jl package to solve the `TigerPOMDP` from the POMDPModels.jl package. Provide the following two deliverables:

a) Plot the alpha vectors from SARSOP[2] and the pseudo alpha vectors from QMDP.

b) Evaluate the QMDP policy and a near-optimal policy calculated with the SARSOP.jl package using Monte Carlo simulations. Report the average return and standard error of the mean.

Question 2. (20 pts)
Evaluate the following three policies on the cancer problem that you created in Homework 5:

1. The QMDP policy obtained using your code from Question 1 above.
2. A heuristic policy *that outperforms the QMDP policy*[3].
3. A near-optimal policy calculated by the SARSOP solver from the SARSOP.jl package.

First, report the results of these simulations in a table, then write a short paragraph answering the following question: Both the tiger and cancer problems are similar in that they have information-gathering actions. However, the performance gap between the QMDP and optimal solutions in one of the problems is much larger. Which problem has the larger gap, and why?

[1] This can be a discrete Bayesian filter or a particle filter - the discrete filter is easier to implement

[2] You can use the `alphavectors` function from POMDPPolicies.jl to access the alpha vectors from the policy that SARSOP returns.

[3] Hint: you may want to use the QMDP policy within your heuristic policy, i.e. take the QMDP actions some of the time.

Question 3. (25 points) Two countries are deciding whether to pursue protectionist (P) or free trade (F) policies. One model focuses on the need to protect key industries for national security. In this model a country receives a payoff of 1 if they protect these industries and a payoff of -1 if only the other country protects them, resulting in the following payoff matrix: (Note: this game was changed slightly from the original homework release.)

	P	F
P	$1, 1$	$1, -1$
F	$-1, 1$	$0, 0$

a) Are there any dominant strategy equilibria in this game? If so, what are they? Briefly justify your answer.

b) Are there any pure Nash equilibria in this game? If so, what are they? Briefly justify your answer.

A second model focuses on comparative advantage and the larger effective size of the economy that results from free trade. In this model a country receives a payoff of 1 if they pursue free trade and a payoff of 1 if the other country pursues free trade. These payoffs add together if both countries pursue free trade. There are no other payoffs in this model.

c) What is the payoff matrix for this model?

d) Are there any dominant strategy equilibria in this game? If so, what are they? Briefly justify your answer.

e) Are there any pure Nash equilibria in this game? If so, what are they? Briefly justify your answer.

A third model that takes into account a variety of factors is the following:

	P	F
P	$8, 8$	$9, 6$
F	$6, 9$	$10, 10$

f) Are there any dominant strategy equilibria in this game? If so, what are they? Briefly justify your answer.

g) Are there any Nash equilibria in this game? If so, what are they? Make sure to include all Nash equilibria and describe the policies used by both players precisely. Breifly justify your answer.

2 Challenge Problem

Question 4. (Lasertag POMDP, 35 pts)

In this problem, you will create a high-performance policy for a laser tag POMDP model `HW6.LaserTagPOMDP()`. In this POMDP, a robot seeks to tag a moving target in a grid world with obstacles. The state space consists of all positions the robot and the target can take, and the problem ends with a reward of 100 as soon as they occupy the same cell. There are five actions, `:up`, `:down`, `:left`, `:right`, and `:measure`. The `:measure` action has a cost of -2 and gives the robot returns from lasers pointed in the four directions indicating the *exact* distance to the first wall, obstacle, or target that the laser encounters. When any other action is taken, the reward is -1 and the laser observations are much less reliable: 90% of the time they fail (returning 0), and 10% of the time they return an accurate distance. The source code, along with the `POMDPs.jl` interface including `POMDPTools.render` can be used to further explore the problem.

a) Submit a `Policy` or a tuple containing a `Policy` and an `Updater` to the `HW6.evaluate` function. A score of 15 will get full credit. You can use or modify your QMDP code from Question 1, or you are encouraged to use any tools available to you - any POMDPs.jl solvers, deep reinforcement learning, a modification of your MCTS code from earlier homework, or heuristic policies are all acceptable. The `Policy` object may be a solution that was calculated offline, or an online planner. Submit the `results.json` file produced by `HW6.evaluate` to Gradescope.[4]

b) Write a short paragraph describing your approach.

[4] The `evaluate` function roughly runs

```
@showprogress for i in 1:n_episodes
    ro = RolloutSimulator(max_steps=100, rng=rng)
    returns[i] = simulate(ro, LaserTagPOMDP(), policy, up)
end
score = sum(returns)/n_episodes
```

You are encouraged to use this and some of the debugging steps you used in homework 3.